

RepoMind

Architecture, Design Decisions, Data Flow & Retrieval Roadmap

A codebase-focused Retrieval-Augmented Generation (RAG) system built from first principles with TypeScript.

Current implementation status

Repository ingestion, language-aware semantic chunking, Markdown block-aware chunking, Voyage embeddings, Chroma Cloud indexing, batch processing, and collection reset are implemented. Hybrid retrieval and prompt construction are the next major stages.

Document purpose: A durable technical reference covering architecture, module responsibilities, design rationale, data flow, current trade-offs, and next steps.

1. Executive Overview

RepoMind is a repository-aware RAG system designed to ingest a GitHub codebase, understand its files according to their language or document type, split the repository into retrieval-friendly chunks, generate embeddings, store those embeddings in Chroma Cloud, retrieve relevant context, and eventually use an LLM to answer questions about the repository with file and line-level references.

The project deliberately avoids hiding the core RAG mechanics behind a large orchestration framework at this stage. Ingestion, chunking, vector indexing, and retrieval are being built directly in TypeScript, with focused libraries used for specific jobs.

The current design favors **clarity and working functionality over premature production optimization**. For the first complete version, a single Chroma collection represents the currently indexed repository. The collection is reset before a new repository is indexed.

High-level pipeline

```
GitHub URL
↓
RepoLoader
↓
Local repository
↓
RepoScanner
↓
CodeFile[]
↓
Language/document-aware chunking
↓
CodeChunk[]
↓
Voyage embeddings
↓
Chroma Cloud
↓
Hybrid retrieval (next)
↓
Prompt construction (next)
↓
LLM (next)
↓
Grounded answer + source references
```

Current milestone

Stage	Status	Purpose
Repository cloning	Implemented	Clone a GitHub repository locally.
Repository scanning	Implemented	Discover relevant files while respecting .gitignore and exclusions.
Semantic code chunking	Implemented	Use Tree-sitter syntax structure instead of arbitrary character splits.
HTML/CSS/JSON chunking	Implemented	Use document-specific structural boundaries.
Markdown chunking	Implemented/optimized	Use headings, hierarchy, logical blocks, and size-aware grouping.
Embedding	Implemented	Use Voyage code embeddings through Chroma's embedding integration.
Chroma indexing	Implemented	Batch-index the full repository.
Collection reset	Implemented	Ensure the active collection represents the current repository.
Vector retrieval	Next	Retrieve semantically relevant chunks.
Hybrid retrieval	Next	Combine semantic and keyword/BM25 signals.

Stage	Status	Purpose
Prompt construction	Next	Turn retrieved context into a grounded LLM prompt.
LLM generation	Pending	Generate the final repository-aware answer.

2. System Architecture

2.1 Architectural principle

RepoMind is organized as a pipeline of focused modules. Each layer has a clear responsibility and passes a well-defined data representation to the next layer. This prevents the entry point from becoming a large monolithic script and makes individual stages testable.

```
src/
  ■■■ index.ts
  ■■■ ingestion/
    ■ ■■■ repoLoader.ts
    ■ ■■■ repoScanner.ts
    ■ ■■■ types.ts
  ■■■ chunking/
    ■ ■■■ astUtils.ts
    ■ ■■■ chunkFile.ts
    ■ ■■■ codeChunker.ts
    ■ ■■■ cssChunker.ts
    ■ ■■■ documentChunker.ts
    ■ ■■■ grammars.ts
    ■ ■■■ htmlChunker.ts
  ■■■ vectorstore/
    ■ ■■■ chroma.ts
    ■ ■■■ indexChunks.ts
    ■ ■■■ testChroma.ts
  ■■■ retrieval/
  ■■■ (hybrid retrieval modules – next stage)
```

2.2 Core data contracts

```
Repository
{
  name: string;
  url: string;
  localPath: string;
}

CodeFile
{
  path: string;
  language: string;
  content: string;
  size: number;
}

CodeChunk
{
  content: string;
  filePath: string;
  language: string;
  startLine: number;
  endLine: number;
  symbol?: string;
  type: string;
}
```

Repository identity is kept separate from CodeChunk. A chunk describes content and its location inside a file. Repository-level information can be supplied by orchestration/indexing when needed. For the current single-active-repository design, repository metadata does not need to be embedded into every chunk.

3. Ingestion Layer

3.1 RepoLoader

Module: `src/ingestion/repoLoader.ts`

Responsibility: accept a GitHub URL, derive a local repository name/path, clone the repository when it is not already present, and return a Repository object.

The loader avoids recloning an already-present repository. This is a local development convenience and keeps repeated pipeline runs fast.

3.2 RepoScanner

Module: `src/ingestion/repoScanner.ts`

Responsibility: discover files, apply ignore rules, identify supported languages, enforce file-size limits, and return CodeFile objects.

- Uses fast-glob for repository-wide file discovery.
- Uses ignore to apply .gitignore rules.
- Explicitly ignores .git and node_modules.
- Skips unsupported file extensions.
- Skips very large files above the configured size threshold.
- Skips common dependency lockfiles so they do not enter language chunkers.

The scanner decides **which files enter the processing pipeline**. Chunkers decide **how an accepted file is segmented**. This prevents repository-specific exceptions from leaking into individual chunkers.

4. Chunking Architecture

Chunking is one of the most important RepoMind design decisions. The project does not use one generic character-based splitter for every file. Different file types have different structures, so the chunking strategy follows the structure of the content.

4.1 Dispatcher

Module: `src/chunking/chunkFile.ts`

The dispatcher selects the appropriate chunker based on the CodeFile language.

Language/type	Chunker	Strategy
JavaScript/TypeScript	codeChunker	Tree-sitter semantic nodes
Python	codeChunker	Functions/classes
Java	codeChunker	Classes, methods, constructors, interfaces, enums
C/C++	codeChunker	Functions, structs/classes
Go	codeChunker	Functions, methods, type declarations
HTML	htmlChunker	Semantic HTML containers
CSS/SCSS	cssChunker	Rules and media statements
JSON	documentChunker	Top-level object sections

Language/type	Chunker	Strategy
Markdown	documentChunker	Heading hierarchy + logical blocks + size-aware grouping

4.2 Tree-sitter and semantic code chunking

Tree-sitter is used because source code is not naturally divided by blank lines or arbitrary character counts. A function, class, method, interface, or type declaration is a meaningful retrieval unit. Tree-sitter provides a syntax tree from which these units can be identified.

The code chunker maps language-specific syntax nodes to CodeChunk objects and extracts names/symbols where the grammar provides them. If a file produces no recognized semantic nodes, a file-level fallback chunk preserves the entire file.

4.3 HTML

HTML is chunked around semantic containers such as header, nav, main, section, article, aside, footer, form, table, script, and style. The implementation preserves original source slices and line ranges. The design intentionally avoids treating every nested HTML element as a separate chunk.

4.4 CSS/SCSS

CSS is structured around rule sets and media statements. A normal selector rule becomes a chunk, while an @media statement remains a coherent chunk containing its nested rules.

4.5 JSON

JSON is treated as structured configuration/data rather than programming-language code. For root objects, top-level key/value pairs become separate chunks, with the key as the symbol. package.json therefore becomes meaningful sections such as dependencies, scripts, engines, and files.

Lockfiles are excluded at scanning time rather than being special-cased inside the JSON chunker. This keeps responsibilities clean.

4.6 Markdown

Markdown required a different approach. It is primarily natural language, so a pure AST-oriented strategy is less useful than structural text chunking. The current design combines heading hierarchy with logical blocks and a target chunk size.

- Headings define semantic sections.
- Heading hierarchy is retained as context, e.g. Installation > Configuration > Environment Variables.
- Paragraphs and other logical blocks are grouped around a target size.
- Fenced code blocks are kept intact and are not split internally.
- Files without headings can still be split by logical blocks.
- A heading/context prefix can be included in chunk content to improve embedding context.
- Tiny Markdown files are not forced into multiple chunks.

5. Chunk Metadata and Retrieval Context

Every CodeChunk carries content and location information. This metadata is deliberately designed for later retrieval and citation.

Field	Meaning	Why it matters
content	Actual chunk text	Embedded and provided to the LLM as context.
filePath	Repository-relative path	Lets answers cite the source file.
language	Detected language/document type	Useful for filtering and prompt context.
startLine	First source line	Supports precise citations.
endLine	Last source line	Supports precise citations.
symbol	Function/class/heading/key/etc.	Improves interpretability and retrieval context.
type	Semantic chunk category	Distinguishes function, class, section, table, JSON section, etc.

Line ranges are particularly important because the eventual answer should be able to point to a concrete file and relevant line range rather than merely saying that information came from the repository.

6. Embedding and Vector Storage

6.1 Chroma Cloud

Package: chromadb

Chroma is the vector database used by RepoMind. The project uses Chroma Cloud rather than running a local Chroma server. The TypeScript integration uses CloudClient and a collection named `repomind`.

6.2 Voyage

Package: @chroma-core/voyageai

Embedding model: voyage-code-4.

The embedding function is integrated with Chroma so RepoMind can provide documents to Chroma without manually managing embedding API calls inside the indexing function.

6.3 Why Chroma + Voyage?

- Chroma provides vector storage and similarity-search interfaces.
- Voyage provides the embedding model used for code retrieval.
- The integration keeps application code relatively small.
- Cloud storage is useful for eventual deployment.
- The project still exposes the important RAG mechanics instead of hiding the whole workflow behind a large framework.

6.4 Batch indexing

The indexer processes CodeChunk objects in batches rather than attempting to insert the entire repository in one operation. The current batch size is 100 chunks.

```
for (let start = 0; start < chunks.length; start += BATCH_SIZE) {
  const batch = chunks.slice(start, start + BATCH_SIZE);
  await collection.upsert({
    ids,
    documents,
```

```
metadatas
});
}
```

Upsert makes repeated development indexing safer because the same IDs can be updated rather than causing duplicate-ID failures.

7. Chroma Collection Lifecycle

The current RepoMind version intentionally uses one collection for the currently active repository. Before indexing a new run, the old collection is deleted and a fresh collection is created.

```
Current repository
↓
scan
↓
chunk
↓
delete old 'repomind' collection
↓
create fresh 'repomind' collection
↓
batch index current chunks
```

This is not the most efficient production architecture because unchanged chunks are re-embedded on every indexing run. It is intentionally simple for the first complete RAG implementation. Later, RepoMind can evolve to incremental indexing, stable chunk IDs, repository namespaces, or repository-scoped filtering.

Design principle: optimize the architecture after the complete pipeline works. The current choice reduces synchronization complexity while the project is still learning and validating the core RAG flow.

8. Packages and Their Roles

Package	Role in RepoMind
simple-git	Clone GitHub repositories into the local workspace.
fast-glob	Discover repository files efficiently.
ignore	Apply .gitignore rules during scanning.
tree-sitter	Parse source files into syntax trees.
tree-sitter-javascript	JavaScript grammar.
tree-sitter-typescript	TypeScript/TSX grammar.
tree-sitter-python	Python grammar.
tree-sitter-java	Java grammar.
tree-sitter-c	C grammar.
tree-sitter-cpp	C++ grammar.
tree-sitter-go	Go grammar.
tree-sitter-html	HTML grammar.
tree-sitter-css	CSS grammar.
tree-sitter-json	JSON grammar.
chromadb	Connect to Chroma Cloud and manage collections/documents/search.
@chroma-core/voyageai	Connect Chroma to Voyage embeddings.

Package	Role in RepoMind
dotenv	Load local environment variables such as API keys and Chroma credentials.

The project deliberately does not depend on LangChain or LlamaIndex for the core pipeline. This is a learning and control decision, not a claim that those frameworks are unsuitable.

9. Detailed End-to-End Data Flow

Stage 1 — URL to Repository

```

User / CLI
↓
GitHub URL
↓
cloneRepository()
↓
Repository { name, url, localPath }

```

Stage 2 — Repository to CodeFile[]

```

Repository.localPath
↓
fast-glob
↓
.gitignore + default ignores
↓
extension/language detection
↓
size checks
↓
CodeFile[]

```

Stage 3 — CodeFile[] to CodeChunk[]

```

CodeFile
↓
chunkFile()
■■■ codeChunker
■■■ htmlChunker
■■■ cssChunker
■■■ documentChunker
■■■ JSON
■■■ Markdown
↓
CodeChunk[]

```

Stage 4 — CodeChunk[] to Chroma

```

CodeChunk[]
↓
batch of 100
↓
Voyage code embedding
↓
vector + document + metadata
↓
Chroma Cloud collection

```

Stage 5 — Retrieval (next)

```

User question
■■■ semantic/vector search
■■■ keyword/BM25 search
↓
result fusion
↓
top relevant chunks

```

Stage 6 — Prompt construction (next)

```
Question
+
retrieved CodeChunks
↓
grounded prompt
↓
LLM
↓
answer + file/line references
```

10. Hybrid Retrieval Design — Next Stage

The next retrieval implementation will combine semantic vector search with lexical keyword search. This is especially valuable for code because some questions are conceptual while others contain exact identifiers, error messages, function names, class names, configuration keys, or API symbols.

Signal	Strong at	Example
Vector search	Conceptual similarity and paraphrased questions	"How does Express process middleware?"
BM25/keyword search	Exact identifiers and terminology	"application.use" or "ERR_HTTP_HEADERS_SENT"
RRF fusion	Combining ranked lists without comparing incompatible raw scores	Promote chunks that rank well in both systems.

Reciprocal Rank Fusion (RRF) is the planned first fusion method. It combines ranking positions rather than directly comparing vector distances to BM25 scores, which are on different scales.

```
RRF(chunk) =
1 / (k + vectorRank)
+ 1 / (k + keywordRank)
```

The retrieval implementation should remain modular: vector search, keyword search, and fusion should be separate functions. This makes each retrieval signal independently testable.

11. Prompt Construction Design — Planned

After retrieval, the system should construct a prompt that clearly separates instructions, repository context, and the user's question.

```
You are RepoMind, a codebase assistant.

Answer the user's question using the provided repository context.

Rules:
- Use repository context as the primary source.
- Do not invent repository behavior.
- If context is insufficient, say so.
- Cite relevant files and line ranges.

Repository context:
[1] lib/application.js:120-145
...
[2] lib/router/index.js:50-82
...

User question:
How does Express handle middleware?
```

The prompt builder should not perform retrieval itself. It should receive already-ranked chunks and format them into a bounded context window.

12. Major Design Decisions and Rationale

Decision	Reason
Raw TypeScript first	Makes RAG mechanics visible instead of hiding them behind a framework.
Functional/module-based architecture	Keeps modules small and avoids unnecessary class-heavy structure.
Tree-sitter for code	Uses syntax-aware boundaries that correspond to meaningful code units.
Different chunkers by content type	Code, HTML, CSS, JSON, and Markdown have different useful boundaries.
Markdown uses text/structure	Markdown is primarily natural language; headings and logical blocks are useful retrieval boundaries.
Chroma Cloud	Avoids local vector-server management and supports eventual deployment.
Voyage code embeddings	Provides the semantic representation used for code retrieval.
Batch size 100	Avoids one huge indexing operation and provides visible progress.
Upsert	Makes repeated development indexing safer.
Reset collection per run	Simple single-repository model; avoids stale chunks and synchronization complexity.
Hybrid retrieval	Combines conceptual semantic matching with exact terminology matching.
RRF fusion	Avoids calibrating incompatible vector and BM25 score scales.

13. Testing and Validation So Far

The project has been validated incrementally rather than only at the end. This has exposed edge cases and produced concrete architectural fixes.

Test	Observation / outcome
Chroma Cloud + Voyage	Successfully indexed and queried a test document, proving the vector-store/embedding integration.
Express repository	Semantic chunking produced thousands of chunks, validating the architecture on a real codebase.
package.json	JSON chunking produced meaningful top-level sections.
HTML/CSS tests	Semantic boundaries and source line ranges worked on real repository files.
Small repositories	Full pipeline exposed lockfile handling and parser edge cases that were fixed or isolated.
Full batch indexing	Repository chunks can be indexed in batches rather than only the first five test chunks.
Collection reset	The architecture was simplified to replace the current collection before a new repository is indexed.

The testing philosophy is to use small repositories for fast iteration and a larger repository such as Express for realistic scale validation.

14. Current Limitations and Trade-offs

- Collection reset means unchanged chunks are re-embedded on every indexing run.
- The current architecture represents one active repository at a time.
- Some Tree-sitter grammars are not currently included, such as YAML.
- HTML semantic containers can be improved further to reduce parent/child overlap in complex documents.
- Markdown chunk size is currently character-based rather than token-based.

- Retrieval and prompt construction are not yet integrated into the final pipeline.
- Hybrid retrieval will require a persistent or reconstructible keyword index; the final storage design should avoid unnecessary duplication of full chunk data.

15. Roadmap After the First Working RAG

Phase	Goal
1	Implement vector retrieval against Chroma.
2	Implement BM25/keyword retrieval.
3	Fuse results with RRF and test retrieval quality.
4	Build prompt construction with context limits and source references.
5	Connect an LLM and complete question → answer.
6	Create evaluation questions and measure retrieval/answer quality.
7	Improve retrieval with reranking, metadata filters, query expansion, or tuning if needed.
8	Replace full re-indexing with incremental indexing when performance matters.
9	Expose the pipeline through an API and frontend.
10	Add multi-repository support and production-grade persistence when required.

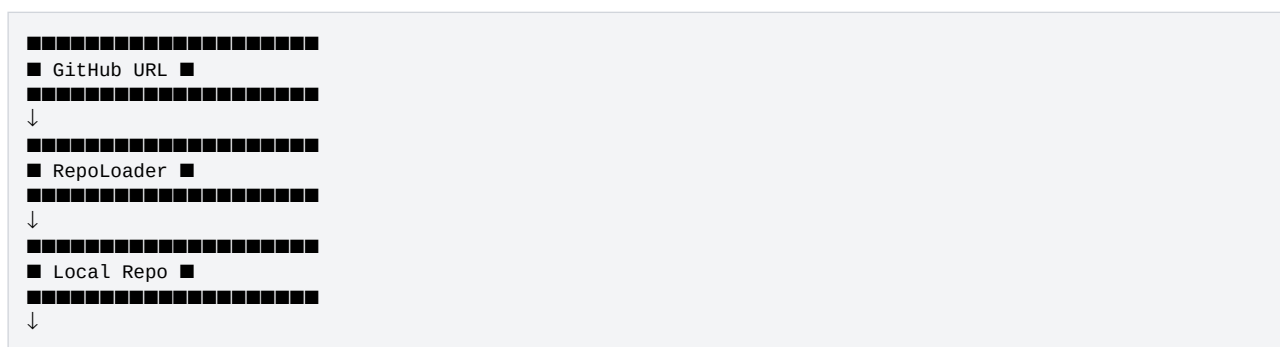
16. Development Philosophy

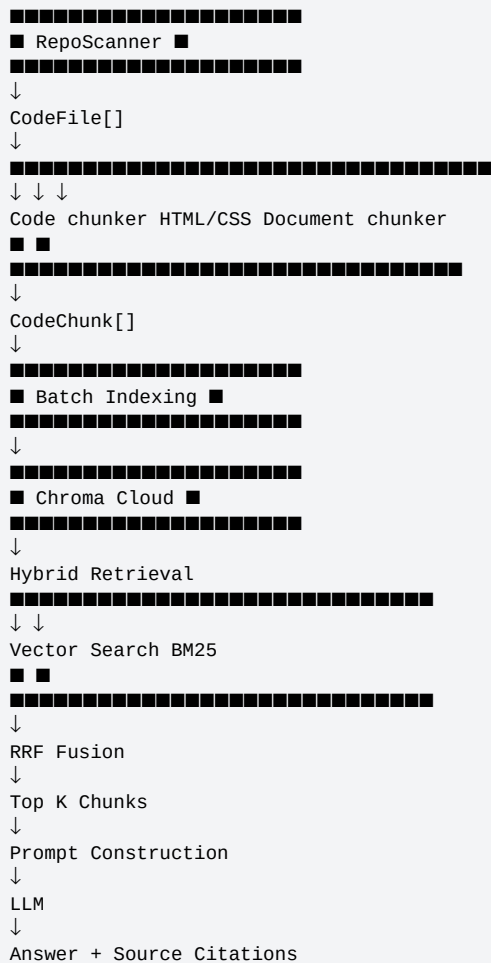
RepoMind intentionally follows the principle: **make it work, understand it, test it, then make it beautiful.**

The first objective is a complete vertical slice. Once a user can ask a repository question and receive a grounded answer based on retrieved source chunks, optimization can happen from measured problems rather than assumptions.

- Do not prematurely optimize indexing.
- Keep retrieval components independently testable.
- Keep source metadata throughout the pipeline so answers can cite files and lines.
- Prefer explicit data flow over hidden framework behavior while learning the architecture.
- Add production complexity only when the project actually needs it.

17. Final Reference Architecture





18. Closing Summary

RepoMind is currently at the transition point between ingestion/indexing and retrieval. The repository can be transformed into structured, language-aware chunks, embedded with Voyage, and stored in Chroma Cloud in batches. The remaining core RAG work is to retrieve the right chunks, combine semantic and lexical evidence, construct a grounded prompt, and connect an LLM.

The architecture intentionally leaves room for later improvements without requiring them now. The current collection-reset strategy, simple module boundaries, and explicit CodeChunk metadata provide a clean learning-oriented foundation. Once the complete RAG pipeline works, the system can be optimized incrementally based on real retrieval and latency observations.